

Inlämning 2

Lösningen och kritisk reflektion

TeacherAid — AI-stöd för lärare på Yrkesakademin

Student: Kim Safsten | Kurs: SYS25D | Medieinstitutet

Inledning

Denna rapport redovisar säkerhetsanalysen och den kritiska reflektionen för TeacherAid — den lösning jag rekommenderade kunden Anna Lindqvist i Inlämning 1. Projektet bygger vidare på beslutsunderlaget: lokal AI (Ollama), RAG mot kursmaterial i PostgreSQL/pgvector, och n8n för att automatisera feedbackutkast på studentinlämningar.

TeacherAid är en RAG-baserad webbapplikation som hjälper lärare att generera feedbackutkast och besvara kursfrågor. All AI-inferens sker lokalt, ingen studentdata lämnar organisationens infrastruktur.

Leverans och åtkomst

Del	Plats i repot
.NET-backend	TeacherAid.Api/
Enhetstester	TeacherAid.Tests/ (19 tester)
n8n-automation	n8n-workflow.json (importeras i n8n på http://127.0.0.1:5678)
Frontend	teacher-aid-frontend/
Körinstruktioner	README.md
Rapport (PDF)	docs/Inlamning2_TeacherAid_KimSafsten.pdf
Rapport (källa)	docs/Inlamning2_TeacherAid_KimSafsten.md
Lösningsbeskrivning	docs/losningsbeskrivning.md
Diagram	docs/flowchart.svg, docs/sequence-diagram.svg, docs/er-diagram.svg

GitHub-repo: <https://github.com/kimsafsten/teacher-aid>

Snabbstart efter klon:

```
docker-compose up -d
cd TeacherAid.Api && dotnet run
```

Se README.md för modellnedladdning, appsettings.Development.json och frontend.

Lösningen i korthet

Lösningen består av fyra delar enligt uppgiften:

- 1. **.NET-backend** — REST API med JWT, RAG (RagService), filsynk (FolderSyncService), pseudonymisering och materialgenerering.
- 2. **Automatisering** — n8n-workflow triggas via webhook när en inlämning ska få AI-feedback; Ollama genererar utkast som sparas i FeedbackDrafts.
- 3. **Säkerhetsanalys** — se Del 1 nedan.
- 4. **Kritisk reflektion** — se Del 2 nedan.

Avvikelser från Inlämning 1

Beslut i Inlämning 1	Implementation	Motivering
Molntjänst diskuterad	Ollama lokalt i Docker	Eliminerar DPA-krav och GDPR-risk vid extern dataöverföring
En kurs i MVP	Backend stödjer courseId; frontend förifyllt med SYS25D	Fokus på fungerande prototyp före fler kurser
AI-feedback som utkast	Human-in-the-loop kvar	Läraren måste godkänna via PUT /api/submissions/{id}/feedback

Designval och AI-användning

Primärt AI-verktyg: Cursor (IDE-integration, hela kodbasen som kontext). **Komplement:** Claude för arkitektur och säkerhetsanalys.

Var AI hjälpte

- **Boilerplate** — controllers, DTO:er, EF Core-migrationer, test-skelett.
- **ILLMService-interface** — Claude föreslog abstraktionen tidigt; gjorde det möjligt att mocka AI-lagret i tester och byta modell utan att röra controllers.
- **n8n-workflow** — snabb prototyp av webhook → Ollama → Postgres utan egen orchestrator-kod.
- **Enhetstester** — AI genererade testfall för TextPseudonymizer, SubmissionFileNameParser och TextChunker; jag valde vilka edge cases som var värda att testa.

Var jag korrigerade AI

Problem	Åtgärd
Raw SQL i n8n bröts av apostrofer i AI-output	Bytte till parameteriserade frågor i Postgres-noden
[AllowAnonymous] på filnedladdning (IDOR)	Borttagen; JWT krävs i API och frontend
studentName skickades till Ollama i webhook	Borttagen ur payload; pseudonymisering i synk
Output-sanitisering (Sanitize) i API-vägen	Implementerad i OllamaLLMService; n8n-vägen saknar ännu motsvarande (Åtgärd Fas 2)
Dubbel EF-migration RemoveAiGrade efter merge	Tog bort tom dublett; behöll migration som faktiskt tar bort AiGrade
EnsureCreated() + Migrate() blandat	Bytte till enbart Migrate()

Del 1: Säkerhetsanalys

Analysen följer kursens mall (V4D1): två lager, OWASP-genomgång, verifierade fynd, klassisk appsäkerhet och integritet.

01 — Översikt: två lager

TeacherAid har två attacktyper som båda granskats.

Lager 1 — Appen (.NET, PostgreSQL, React): JWT-inloggning för lärare, filsynk från mappar, REST-endpoints, databaslagring av inlämningar och feedbackutkast. Här gäller klassisk appsäkerhet: behörighet, IDOR, hemligheter i konfiguration.

Lager 2 — AI-delen (Ollama, n8n, RAG): Elevtext och kursfrågor skickas till llama3; RAG hämtar chunks från pgvector; n8n orkestrerar webhook → Ollama → Postgres. Här tillkommer LLM-risker: prompt injection, felaktig output, informationsläckage.

AI-delen sitter i n8n-workflödet (n8n-workflow.json), OllamaLLMService (RAG och materialgenerering) och den anonyma Q&A-endpointen `POST /api/qa/ask`.

Genomförda åtgärder (Åtgärd Fas 1)

Efter initial analys implementerades följande i koden:

#	Åtgärd	Fil / komponent	Effekt
1	[AllowAnonymous] borttagen från filnedladdning	SubmissionsController.GetFile()	Stänger IDOR — filer kräver JWT
1b	Autentiserad nedladdning i frontend	SyncPanel.DownloadFile()	Skickar token vid filhämtning
2	studentName borttagen ur n8n-payload	FeedbackWebhookTrigger	Elevnamn skickas inte längre till Ollama
3	Avgränsare i n8n-prompten	n8n HTTP Request-nod	###STUDENT_TEXT_START/END### + instruktion att ignorera elevdata
4	Lösenord flyttat till config	AuthService, appsettings.Development.json	Inga hårdkodade credentials i källkod
5	UI-varning vid godkännande	SyncPanel.jsx, FeedbackView.jsx	Påminner läraren att granska AI-utkast innan sparning

02 — OWASP LLM Top 10 (2025)

Risk	Relevant?	Motivering
LLM01 Prompt Injection	Ja	Elevtext i inlämning skickas till Ollama via n8n i samma prompt som systeminstruktioner.
LLM02 Sensitive Information Disclosure	Delvis	Elevdata lagras lokalt; pseudonymisering och lokal Ollama begränsar extern läcka.
LLM03 Supply Chain	Nej	Standard NuGet/npm/Docker; ingen extern AI-API i produktionsflödet.
LLM04 Data and Model Poisoning	Nej	Ingen egen modellträning; endast inloggad lärare kan synka kursmaterial.
LLM05 Improper Output Handling	Ja	n8n sparar rå Ollama-svar i databasen; API-vägen saniterar med Sanitize().
LLM06 Excessive Agency	Nej	AI skapar enbart utkast; läraren godkänner explicit innan sparning.
LLM07 System Prompt Leakage	Delvis	Anonym Q&A kan exponera kursmaterial via RAG, inte elevdata eller systemprompt.
LLM08 Vector and Embedding Weaknesses	Delvis	Embeddings lagras i pgvector; databasen är lokal och inte internetexponerad.
LLM09 Misinformation	Ja	llama3 (7B) kan ge sakligt felaktig feedback som läraren godkänner utan granskning.
LLM10 Unbounded Consumption	Nej	Lokal Ollama utan per-token-kostnad; resursen begränsas av serverns CPU/GPU.

Prioritering (riskmatris)

Score = sannolikhet × konsekvens (1–5). HÖG ≥12, MEDEL 5–11, LÅG ≤4. *Efter Åtgärd Fas 1.*

Risk	S × K	Score	Nivå
LLM09 Misinformation	4 × 5	20	HÖG
LLM05 Improper Output Handling	3 × 3	9	MEDEL
LLM01 Prompt Injection	2 × 4	8	MEDEL
LLM08 Vector/Embedding	2 × 3	6	MEDEL
LLM02 Sensitive Info Disclosure	1 × 4	4	LÅG
LLM07 System Prompt Leakage	2 × 2	4	LÅG
Övriga (LLM03–04, 06, 10)	—	≤3	LÅG

03 — Fynd (OWASP LLM)

De tre allvarligaste kvarvarande riskerna efter Åtgärd Fas 1.

Fynd 1 — LLM09: Felaktig AI-feedback

Vad	llama3 (7B) kan generera sakligt felaktig feedback som läraren godkänner utan granskning.
Attackväg	Inlämning → n8n → Ollama → felaktigt utkast → läraren klickar "Godkänn och spara" → eleven får fel återkoppling.
Befintlig kontroll	Human-in-the-loop, AI-utkast i UI, UI-varning "Granska alltid — AI kan ha fel" , AutomationLog, assignmentDescription och gradingRubric i webhook-payloaden.
Åtgärd	Validera RAG-index. Flagga utkast som avviker från förväntat format.
Integritet	Felaktig feedback kan vara orättvis mot eleven. AI sparar tid på utkast men ersätter inte lärarens professionella bedömning.

Fynd 2 — LLM01: Prompt injection via studentinlämning

Vad	Elev kan skriva instruktioner i inlämningen som försöker manipulera AI-modellen.
Attackväg	"Ignorera alla instruktioner..." i inlämning → n8n → Ollama → missvisande positiv feedback.
Befintlig kontroll	Åtgärd Fas 1: avgränsare ###STUDENT_TEXT_START/END### . Pseudonymisering. Human-in-the-loop. Inget betygsförslag i prompt.
Åtgärd	Åtgärd Fas 2: webhook-hemlighet. Flagga avvikande output. Ev. detektera misstänkta mönster i input.
Integritet	Strikt filtrering kan ta bort legitima formuleringar i kodinlämningar — avgränsare valdes framför hård filtrering.

Fynd 3 — LLM05: Osaniterad AI-output i n8n-flödet

Vad	n8n sparar Ollamas råsvar direkt i PostgreSQL utan sanitisering.
Attackväg	Null-bytes eller kontrolltecken i AI-svar → DB-fel eller trasigt UI vid visning.
Befintlig kontroll	OllamaLLMService.Sanitize() i API-vägen (RAG, materialgenerering). Parametriserad SQL i n8n.
Åtgärd	Åtgärd Fas 2: Code-nod i n8n före Postgres-insert, eller flytta feedbackgenerering till API.
Integritet	Krasch i feedbackflödet kan innebära att vissa elever inte får återkoppling — differentialbehandling.

04 — Klassiska risker

Fynd 4 — IDOR: oautentiserad filnedladdning [åtgärdad i Åtgärd Fas 1]

Vad	GET /api/submissions/{id}/file hade [AllowAnonymous] — obehörig kunde enumerera ID och ladda ner elevinlämningar.
Attackväg	Gissa eller iterera submission-ID → hämta fil utan JWT.
Befintlig kontroll	[AllowAnonymous] borttagen; endpoint kräver JWT. Frontend skickar token via SyncPanel.downloadFile().
Åtgärd	Implementerat (Åtgärd Fas 1).

Fynd 5 — Hemligheter i källkod [åtgärdad i Åtgärd Fas 1]

Vad	Lösenord hårdkodat i AuthService.cs.
Attackväg	Läckt repo eller binär → inloggningsuppgifter till lärarkonto.
Befintlig kontroll	Credentials i appsettings.Development.json (gitignorerad).
Åtgärd	Implementerat (Åtgärd Fas 1). Produktion: miljövariabler eller secrets manager.

Fynd 6 — Oskyddad n8n-webhook [kvarstår]

Vad	POST http://127.0.0.1:5678/webhook/feedback saknar autentisering.
Attackväg	Lokal eller nätverksåtkomst → trigga falsk feedbackgenerering eller överbelasta Ollama.
Befintlig kontroll	Webhook lyssnar på localhost; inget exponerat mot internet i MVP.
Åtgärd	Delad hemlighet i X-Webhook-Secret-header, valideras i n8n (Åtgärd Fas 2).

05 — Integritet och etik

TeacherAid hanterar **studentinlämningar** (namn i filnamn och text) och **kursmaterial**. Huvudavvägningen i Inlämning 1 var lokal inferens (Ollama) istället för extern AI-API, ingen elevdata lämnar organisationens infrastruktur och inget DPA mot molntjänst krävs i MVP.

Human-in-the-loop är medvetet valt: AI ger utkast, läraren godkänner. Det minskar risken för automatisk orättvis bedömning (LLM09) men förutsätter att läraren faktiskt granskar.

Pseudonymisering (TextPseudonymizer, avgränsare i prompt) balanserar integritet mot kodkvalitet, hård filtrering av elevtext kan ta bort legitima programmeringskonstruktioner.

Anonym Q&A (POST /api/qa/ask) sänker tröskeln för elever men accepterar att kursmaterial kan citeras via RAG; det är inte elevdata som exponeras.

Del 2: Kritisk reflektion

Jag hade två veckor på mig att bygga TeacherAid från beslutsunderlag till fungerande produkt. Nedan reflekterar jag över vad som fungerade, vad som inte fungerade, och vad jag skulle göra annorlunda, både i det aktuella projektet och i AI-assisterad systemutveckling generellt.

Vad fungerade

RAG-flödet fungerade bättre än förväntat. Att kombinera pgvector direkt i PostgreSQL med Ollama lokalt visade sig vara ett praktiskt val — ett enda Docker-stack hanterar databas, vektorsökning och AI-inferens utan externa beroenden.

ILLMService-**abstraktionen** var det enskilt bästa arkitekturbeslutet. Jag kunde mocka AI-lagret i tester och byta modell utan att röra controllers. Claude föreslog mönstret; det visade sig korrekt.

Lokal AI eliminerade GDPR-komplexiteten. I Inlämning 1 beskrev jag DPA-krav vid externa AI-tjänster. Lokal inferens tog bort det kravet från MVP — ett designval som krävde juridisk förståelse, inte bara kod.

n8n för automationen gjorde det enkelt att prototypa webhook → Ollama → SQL. När apostrofer i AI-output bröt raw SQL hittade jag felet i n8n-noden och fixade med parameteriserade frågor — något AI missat men jag fångade i testning.

Enhetstesterna tvingade fram bra design. Att skriva 19 tester för TextPseudonymizer, SubmissionFileNameParser och TextChunker krävde att logiken extraherades till egna klasser — lättare att testa och byta ut.

Vad fungerade inte

AI-genererad kod behövde mer granskning på affärslogiken. Felhantering och edge cases saknades i första utkast — t.ex. ohanterat undantag när Ollama var under uppstart istället för 503.

Chunking-strategin är naiv. TextChunker.Chunk() splittar enbart på radbrytningar; långa stycken utan \n ger dåliga RAG-chunks.

Tre chunks per fråga oavsett relevans är trubbigt. Behöver hitta en annan med effektiv metod som gett mer konsistenta svar.

Gränssnittets UX kom i kläm. Feedback visas inte direkt kopplat till inlämningen, kursmaterial kan inte hanteras i gränssnittet.

Säkerhet och robusthet hann inte ikapp överallt. Sanitize() finns i API-vägen men inte i n8n-flödet. Webhooken saknar autentisering. Ollama-timeout ger ohanterat undantag. Säkerhetsfixar kan försvinna vid merge/linter om de inte granskas systematiskt.

Kända begränsningar

Listan är synkad med docs/losningsbeskrivning.md.

Arbetsflöde och UX

- **Feedback visas inte på inlämningen** — utkastet finns i granskningsvyn men inte kopplat till originalfilen.

- **Kursmaterial kan inte redigeras i gränssnittet** — synk indexerar, men inget sätt att uppdatera eller ta bort via frontend.
- **Läraren kan inte ladda upp filer i gränssnittet** - alla filer läggs in i mappar i rotfilen utan validering och kontroller.
- **Ingen regenerering i UI** — `POST /api/submissions/{id}/process` reserverad i API (t.ex. om n8n misslyckas) men ingen knapp i `SyncPanel` ännu.

RAG och prestanda

- **Chunking delar inte på meningar** — `TextChunker.Chunk()` splittar enbart på radbrytningar.
- **Endast tre chunks per fråga** — alltid tre närmaste chunks oavsett relevans.
- **Embeddings cachas inte** — nytt embedding-anrop till Ollama per fråga och chunk.

Säkerhet (kvarstår / Tier 2)

- **Oskyddad n8n-webhook** — `POST /webhook/feedback` saknar autentisering.
- **Osaniterad AI-output i n8n-vägen** — API-vägen använder `sanitize()`, n8n-vägen inte.
- **Ingen felhantering för Ollama-timeout** — ohanterat undantag vid uppstart.

Medvetna avgränsningar (by design)

- **En kurs i frontend** — backend stödjer `courseId`, UI förfyllt med `SYS25D`.
- **Elever loggar inte in** — anonyma kursfrågor via `POST /api/qa/ask`.

Vad jag skulle göra annorlunda

Börja med säkerheten, inte funktionaliteten. OWASP-analys och sanitisering borde varit del av kontraktet från start, inte efterkonstruktion.

Använt migrationer konsekvent från dag ett — aldrig blanda `EnsureCreated()` och `Migrate()`.

Skrivit integrationstester — enhetstester täcker hjälpklasser men inte att `SyncController` synkar filer eller att n8n-webhooken tar emot korrekt payload.

Investerat i chunking tidigt — RAG-kvaliteten är ett tak för hela systemet.

När AI är rätt verktyg — och när det inte är det

En vanlig uppfattning är att AI alltid är okej så länge man använder det som stöd och granskar resultatet. Det stämmer delvis, men är inte hela bilden. Det avgörande är inte om man kontrollerar resultatet, utan vad AI faktiskt ersätter.

I mitt projekt `TeacherAid` används AI för att generera feedbackkast på studentinlämningar och svara på kursfrågor via ett RAG-system. Här är AI rätt verktyg – det avlastar läraren från repetitivt arbete. När en lärare läst många inlämningar om samma ämne blir det svårare att hålla dem isär och ge likvärdig återkoppling; AI tröttnar inte och blandar inte ihop eleverna med varandra. Men om läraren skickar feedbacken utan att läsa den har AI:n tagit över ett ansvar som måste ligga hos en människa. Betyg och omdömen är lärarens beslut, inte ett automatgenererat förslag.

Samma princip gäller i själva programmeringen av projektet. AI är ett bra verktyg när man behöver ett snabbt kodutkast, fastnar på syntax eller vill utforska hur man kan strukturera en lösning – förutsatt att man förstår och granskar det man får. Det blir fel när man kopierar kod man inte förstår, eller granskar säkerhetskritisk kod

slarvigt. AI kan generera kod som ser korrekt ut men innehåller subtila fel.

Gemensamt för båda fallen: AI fungerar bäst som förstärkare, inte ersättare. Det sköter det tidskrävande och repetitiva – men den mänskliga bedömningen måste finnas kvar där det faktiskt spelar roll.

Nästa steg (om projektet fortsätter)

MVP levererar kärnflödet (synk, AI-utkast, RAG-frågor, human-in-the-loop). Nedan är prioriterade steg där några medvetet lämnades utanför första versionen — se `docs/losningsbeskrivning.md`.

1. **Åtgärd Fas 2-säkerhet** — webhook-hemlighet och sanitisering av AI-output i n8n.
2. **Robusthet** — meningsfull felhantering vid Ollama-timeout (t.ex. 503 istället för ohanterat undantag).
3. **Integrationstester** — synk av filer och korrekt webhook-payload mot test-databas med mockad Ollama.
4. **UX i lärargränssnittet** — filuppladdning, feedback kopplad till inlämning, regenereringsknapp (`/process`).
5. **RAG** — bättre chunking och relevansfiltrering (idag: tre chunks utan tröskel; embeddings cachas inte).
6. **Bedömningsfeedback per kurskriterier** — utöka feedback så AI visar hur varje elev uppnått respektive kriterium i bedömningsmallen. Viktig funktion från Inlämning 1 som medvetet låg utanför MVP men är nästa stora steg för kundvärdet.